

Capítulo 3

Herencia

Uno de los pilares de la Programación Orientada a Objetos es la **Herencia**. La herencia la vimos brevemente en el Capítulo 2, no obstante entraremos más a detalle en este capítulo.

Herencia

La herencia es un principio fundamental de la Programación Orientada a Objetos (POO) que permite a una clase (llamada subclase o clase hija) heredar propiedades y métodos de otra clase (conocida como superclase o clase padre).

La herencia permite definir una clase en términos de otra, lo cual hace más fácil el mantenimiento y desarrollo de una aplicación. También proporciona una oportunidad para la reutilización de código y con ellos tener un desarrollo más rápido. Cuando una clase X hereda de otra clase Y, la clase X hereda todos sus atributos y comportamiento. La clase X se conoce como clase hija o clase derivada y la clase Y como super clase o clase padre.

3.1. Tipos de herencia

La herencia se puede clasificar en:

-
1. Simple (Figura 3.1a)
 2. Multi-nivel (Figura 3.1b)
 3. Jerárquica (Figura 3.1c)
 4. Múltiple (Figura 3.1d)

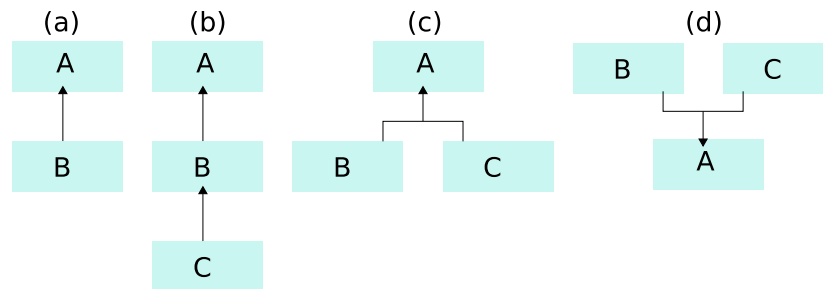


Figura 3.1: Tipos de herencia

3.1.1. Herencia simple

En esta herencia una clase simplemente hereda de otra. La Herencia simple en Java se puede representar como se muestra en el ejemplo del Código 3.1.

```
1 class Automovil {  
2  
3 }  
4 class Sentra extends Automovil {  
5  
6 }
```

Código 3.1: Herencia simple en Java.

3.1.2. Herencia multi-nivel

En esta herencia hay diferentes niveles (descendientes). La Herencia multi-nivel en Java se puede representar como se muestra en el ejemplo del Código 3.2.

```
1 class Automovil {
2
3 }
4
5 class Rio extends Automovil {
6
7 }
8
9 class RioHatchback extends Rio {
10
11 }
```

Código 3.2: Herencia multi-nivel en Java.

3.1.3. Herencia jerárquica

En esta herencia al menos dos clases en un mismo nivel heredan de otra. La herencia jerárquica en Java se puede representar como se muestra en el ejemplo del Código 3.3.

```
1 class Automovil {
2
3 }
4
5 class Rio extends Automovil {
6
7 }
8
9 class Sentra extends Automovil {
10
11 }
```

Código 3.3: Herencia jerárquica en Java.

3.1.4. Herencia múltiple

En esta herencia una clase hereda de 2 o más clases. En Java no es posible la herencia múltiple debido a un problema de ambigüedad. Considere el ejemplo de la Figura 3.2. La clase A hereda de la clase B y C sus comportamientos. ¿Cuál método f hereda A?

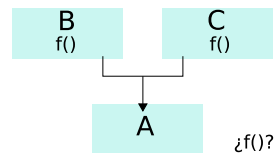


Figura 3.2: Ambigüedad en la herencia múltiple.

La herencia en el lenguaje de programación Java se puede realizar de 3 formas en general:

1. Una clase hereda de una clase.
2. Una clase implementa una interfaz.
3. Una interfaz hereda de otra interfaz.

3.2. Interfaces en Java

Una interfaz es una especie de plantilla para la construcción de clases. Normalmente una interfaz se compone de un conjunto de declaraciones de cabeceras de métodos (sin implementar) que especifican un protocolo de comportamiento para una o varias clases. Además, una clase puede implementar una o varias interfaces. El ejemplo del Código 3.4 muestra la definición de una *interface* en Java. Como se observa, se tienen los métodos sin implementar. Una clase al utilizar la interface esta obligada a implementar sus métodos (ver Código 3.5). Para implementar una interface una clase en Java utiliza la palabra reservada *implements*.

```
1 public interface Clinica {  
2  
3     void registrarPaciente(Paciente e);
```

```
4  Paciente eliminarRegistro(int expediente);
5  void mostrarPacientes();
6  int getTotalPacientes();
7  void citasEdad(int inicio , int fin);
8  void mostrarPorEspecialidad(Especialidad e);
9  void mostrarAgrupacion();
10 void modificarPaciente(int expediente);
11 }
```

Código 3.4: Ejemplo de una interface en Java.

```
1 public class Pegaso implements Clinica{
2
3     public Pegaso() {
4
5     }
6
7     @Override
8     public void registrarPaciente(Paciente e) {
9
10    }
11
12    int buscarPorExpediente(int expediente) {
13        return -1;
14    }
15
16    @Override
17    public Paciente eliminarRegistro(int expediente) {
18        return null;
19    }
20
21    @Override
22    public void mostrarPacientes() {
23
24    }
```

```
24     }
25
26     @Override
27     public int getTotalPacientes() {
28         return -1;
29     }
30
31     @Override
32     public void citasEdad(int inicio , int fin) {
33
34     }
35
36     @Override
37     public void mostrarPorEspecialidad(Especialidad es) {
38
39     }
40
41     @Override
42     public void mostrarAgrupacion() {
43
44     }
45
46     @Override
47     public void modificarPaciente(int expediente) {
48
49     }
50
51 }
```

Código 3.5: Implementación de la interface Clínica.